

ProofPrep: Prompt Log

Every prompt given to Claude Code while building ProofPrep (PromptWars × GenAI Innovation Challenge), in order. Pasted briefs are reproduced verbatim. Where I asked a multiple-choice question, your selected answer is listed under **Your choice**.

Redacted: a Firebase service-account private key (prompt 10) and a Gemini API key (prompt 15) were pasted in chat. They are replaced with [REDACTED] here. Rotate both keys after the hackathon.

1. Project setup and planning

Message: create a project folder called "PromptWars"

Pasted brief:

You are a senior product architect and full-stack engineer helping me plan a hackathon application. Do NOT write any code yet. Your job in this step is to produce a clear, buildable plan.

Context

Hackathon: PromptWars x GenAI Innovation Challenge. Problem statement: "Reimagine Placement Preparation" — help students understand what to prepare, how to prepare, where they stand, and what to improve for campus placements (Indian engineering colleges: aptitude, coding/DSA, CS fundamentals like OS/DBMS/CN/OOP, HR and technical interviews).

Build window: [FILL IN, e.g. 24 hours]. Team size: [FILL IN]. Tech constraints: [FILL IN, e.g. must use Gemini / Google Cloud / Firebase].

Core idea

An AI placement-prep app built on one loop: diagnose → target → practice → re-measure. It should not be a generic prep platform. Its three differentiators are:

1. Mistake Diagnosis — every wrong answer is classified as a concept gap, application gap, careless/time-pressure error, or overconfidence (the student rates confidence before answering). The roadmap adapts to the *type* of mistake, not just the topic.
2. Project Defense — the app reads the student's GitHub repos and resume, checks resume claims against evidence in the code, and runs a mock interview that questions them on their own projects.
3. Communication Coach — spoken answers are transcribed and scored on structure (STAR / claim-reason-example), relevance, filler words, pace and length (never accent). The AI restructures the student's own answer, and the student re-records and compares.

User flow (already decided)

- Login with Google Sign-In (Firebase Auth).
- Three user states: new, partially onboarded (resume where they left off), fully onboarded (go straight to Home).
- Onboarding, required: name, target domain/role, target companies, placement timeline, hours available per day, interests.
- Onboarding, optional (can be added later from Profile): GitHub, LinkedIn, LeetCode, resume upload.
- Onboarding ends with a Quick Assessment (coding, aptitude, CS fundamentals, communication). If skipped, Home shows a single "Take your assessment" call to action instead of empty modules.
- Home dashboard: Placement Readiness Score per target company as the headline, then Skill Gap Analysis, Personalized Roadmap, Daily Practice, Mock Interview, Projects (Project Defense), Communication Coach, and Progress.
- Loop: Daily Practice and Mock Interview results trigger re-assessment, which updates the Skill Gap, the Roadmap and the Readiness Score.

What I need from you

1. Critique the flow: gaps, edge cases, or unnecessary steps. Be direct.
2. Screen list: every screen, its purpose, and its key UI elements.
3. Data model: the main entities (User, Profile, Assessment, Attempt, Mistake, Skill, Roadmap, Task, Interview Session, Project, Communication Attempt, Readiness Score) with fields and relationships, designed for Firestore.
4. Module dependency map: what data each module reads and writes, and what triggers each update.
5. Readiness Score formula: how it is calculated per target company, with transparent weights a judge could verify.
6. AI prompt chain: every point where an LLM is called, with the input, output format (JSON schema) and purpose of each prompt. Examples include the claim extractor, repo analyzer, interviewer, answer evaluator, mistake classifier, roadmap generator and communication scorer.

7. Recommended tech stack, with a one-line reason for each choice.
8. Scope split: MVP (must work in the demo), nice-to-have, and stretch. Keep the MVP achievable within the build window.
9. Build order: a step-by-step sequence for building the MVP.

Before answering, ask me up to 3 clarifying questions if anything critical is missing. Otherwise, proceed. Use clear headings and keep each section concise.

Your choice: Build window: 24 hours · Team size: 3–4 people · Tech constraints: none.

2. Phase 0: foundation

You are a senior full-stack engineer. We are building "ProofPrep", an AI placement-prep app, in a 24-hour hackathon with a team of 4. The full product plan is in PLAN.md at the repo root. Read it fully before doing anything. Treat it as the source of truth; if you think something in it is wrong, say so instead of silently changing it.

This phase: Foundation only (PLAN.md Section 9, "Hours 0–2")

Build ONLY the shared foundation that four parallel tracks will build on. Do not build any screens or features beyond what is listed here.

Deliverables

1. Scaffold: Next.js (App Router) + TypeScript (strict) + Tailwind + shadcn/ui. ESLint + Prettier. Folder structure:

- app/ (routes)
- components/ (UI)
- lib/firebase/ (client.ts, admin.ts)
- lib/llm/ (call.ts, prompts/)
- lib/engine/ (pure logic: mastery, readiness, classifier rules)
- lib/schemas/ (Zod schemas + inferred types)
- data/seed/ (skills, companies, questions)
- scripts/ (seeding)

2. Firebase: client SDK init (Auth + Firestore + Storage) and Admin SDK init for server routes. Read all config from environment variables. Create .env.example listing every variable. Never hardcode keys.

3. Schemas: Zod schemas and exported TypeScript types for EVERY entity in PLAN.md Section 3, matching the field names exactly. Also Zod schemas for the output of prompts P1–P9 in Section 6.

4. LLM wrapper: lib/llm/call.ts exporting

llm.call<T>(promptId, input, schema: ZodSchema<T>, opts?: { model?: "strong" | "fast" }): Promise<T>

- Provider-agnostic interface. Implement a Claude provider now, using structured output via tool use. Keep the provider behind an interface so a Gemini provider can be added in one file.

- Validate the output with the Zod schema. On a validation failure, retry once with the validation error appended.

Then throw a typed error.

- Log promptId, latency and success/failure to the console.

- Prompts live in lib/llm/prompts/<promptId>.ts as template functions. Create empty stubs for P1–P9 with TODO comments only.

5. Seed data:

- data/seed/skills.ts: the ~25-skill taxonomy across aptitude, dsa, cs and communication, with prerequisites. Propose it; we will review.

- data/seed/companies.ts: the 5 company profiles from PLAN.md Section 5, with exact weights and bars. Assert that the weights sum to 1.

- scripts/seed.ts: writes skills and companies to Firestore with the Admin SDK. It must be idempotent.

6. A placeholder home page that confirms Firebase is connected. Deployable to Vercel.

Rules

- Do not add any dependency not implied by PLAN.md Section 7 without asking me first.
- No features, screens or auth flows yet. Those belong to later phases.
- Keep functions small and typed. No `any`.

When you finish, give me:

1. A list of every file you created and its purpose, one line each.
2. The exact commands to install, seed and run locally.
3. Any decisions you made that PLAN.md did not specify, so I can approve them.
4. Anything in PLAN.md that looks inconsistent or risky.

3. Seeing the site

ok so how do i see the website u made ?

4. Pre-split decisions

Read PLAN.md and README.md. Phase 0 is complete. Before the team splits into parallel tracks, apply these approved decisions and update PLAN.md so it stays the source of truth. Change schemas, seeds and docs only; do not build features.

1. Readiness confidence becomes three levels: "low" | "medium" | "high".
 - aptitude/dsa/cs: low < 6, medium 6–9, high ≥ 10 attempts in the category.
 - projects: medium = ≥1 analyzed project; high = ≥1 analyzed project AND ≥1 completed project_defense session.
 - communication: medium = 1–2 attempts; high = ≥3.
 - Overall confidence = the lowest level among components with weight ≥ 0.15.
2. Add to SkillState: levelStats: { concept: { attempts, correct }, application: { attempts, correct } }, both defaulting to 0.
3. Questions are server-only. Define a PublicQuestion schema: Question minus correctIndex, explanation and distractorTags. Document that clients receive only PublicQuestion via API routes.
4. Add a CommPrompt schema and seed file: { id, text, kind: "behavioral" | "technical" | "opinion", targetDurationSec, skillId? }. Seed 10 prompts: 4 behavioral, 4 technical, 2 opinion. Add them to seed.ts and to the firestore.rules client-read list.
5. CommAttempt gets `overall: number` (0–100) = mean(structure, relevance, clarity, length) × 20. It is computed in code, never by the LLM. createdAt is required.
6. Projects component, as written in PLAN.md Section 5:
projectScore = 50·(supported + 0.5·partial)/totalClaims + 10·defenseScore, where defenseScore comes from the latest completed project_defense session for that project (0 if none). Component S_projects = mean of the top 2 projectScores (0 if there are no projects).
7. Attempt.confidence becomes nullable. In PLAN.md Section 6 P4, state that the overconfident rule applies only when confidence is non-null.

Then:

- Run typecheck, lint, build and seed:check, and fix any failures.
 - Update the PLAN.md sections affected (3, 5, 6) and add a short "Changelog" section at the end.
 - Report the files changed and anything that broke.
-

5–11. Running it and connecting Firebase

5. now how to run it

6. *[Screenshot of the placeholder page showing two red Firebase status lines]* what should i do

7. *[Screenshot of Firebase Project settings]* after this

8. *[Screenshot of the Firebase web app SDK configuration]*

9. *[Pasted Firebase Admin sample code]* this is what i got .

```
var admin = require("firebase-admin");
var serviceAccount = require("path/to/serviceAccountKey.json");
admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});
```

10. *[Pasted the Firebase service-account JSON: [REDACTED] , it contains the private key]* i will delete later for now have it

11. ok done check

12. Track briefs A, B and C (pasted together)

Track A — App shell. You own app/(auth), app/onboarding, app/home, app/readiness, components/layout.

1. Google Sign-In with Firebase Auth, plus a "Try demo account" button (sign in as a fixed demo user; the account itself gets seeded later).
 2. Three-state routing middleware based on users.onboardingStep: new → onboarding, partial → resume at the saved step, done → /home.
 3. Onboarding stepper per PLAN.md Section 1, point 1 (reduced fields). Save each step through an API route so users can resume.
 4. Home layout (Section 2, screen #6): readiness headline per company with its confidence label, Today card, module grid. Use typed mock data behind one getHomeData() function so Track B can swap in real data later.
 5. Readiness detail screen (#7): component bars against company bars, weights table, arithmetic shown, biggest lever.
- Acceptance: a new Google user signs in → onboards → lands on Home with mock data. Signing out and back in skips onboarding. Quitting mid-onboarding and returning resumes at the same step.

Track B — Core engine. You own lib/engine, app/api/assessment, app/api/practice, data/seed/questions.

1. lib/engine/readiness.ts: a pure function implementing PLAN.md Section 5, with the 3-level confidence and the projects rule. Unit tests first: the worked example must produce TCS = 91.7 and Amazon = 69.9, with DSA as Amazon's biggest lever. Also test the edge cases: no data, no projects, a component over its bar.
2. lib/engine/mastery.ts: the Section 4 update rule, including careless → outcome 0.6 and levelStats updates. Unit tested.
3. lib/engine/classifierRules.ts: the P4 rules as a pure function returning a verdict or null. Unit tested for each type.
4. Question bank: seed about 150 questions in data/seed/questions (aptitude, CS, and code-reasoning types). Mark each one reviewed: false; humans verify the answer keys.
5. API routes: start/resume the assessment, get the next PublicQuestion, submit an answer (server-side grading → Attempt → mastery update), finish a section.

Acceptance: all engine tests pass; a scripted run of the full assessment writes correct attempts and skillStates to Firestore.

Track C — Project Defense. You own app/projects, app/interview, app/api/projects, app/api/interview, and prompts P1, P2, P3, P6, P7.

1. Resume upload to Firebase Storage → P1 parser → store the parsed projects and claims.
2. GitHub fetch with Octokit (server token): skip forks; key-file picker (entry points, largest source files, routes, models; max 8 files); file tree to depth 3.
3. P3 repo analyzer → claims table UI (claim → verdict → evidence file links), Section 2 screen #12.
4. Defense interview: P6 once per turn, unsupported claims asked first, maximum 6 questions; P7 evaluation at the end → report screen (#14); write defenseScore back to the project.
5. Fallbacks: no GitHub → resume-only interview with claims marked "unverified"; GitHub errors are shown to the user, never crash the page.

Write the prompts carefully and version them in lib/llm/prompts; they are judged.

Acceptance: upload a real resume + a real repo → the claims table shows at least one verdict with file references → complete a 6-question interview → get a report.

Your choice: Build all three in order · Allowed dependencies: vitest (Track B tests), octokit (Track C).

13. Shared layer + all tracks + integration (one message)

Pasted: shared layer

Read PLAN.md, README.md and lib/schemas. Phase 0/0.5 is done. We have 2 hours to finish and must keep Firebase (Auth + Firestore). Build only the shared layer that four parallel tracks will use:

1. firestore.rules: users may read/write only users/{uid}/** for their own uid. skills, companies and commPrompts are readable by all. questions stays server-only. Deploy-ready.
 2. lib/auth.tsx: an AuthProvider using Firebase Google Sign-In, with a useAuth() hook returning { user, loading, signIn, signOut }.
 3. lib/data/: small typed repository functions using the existing Zod schemas, for example getUserDoc, updateProfile, saveAttempts, saveMistakes, saveCategoryScores, saveReadinessSnapshot, getProjects/saveProject, saveInterviewSession, saveRoadmap/updateTask. Validate data on read with Zod.
 4. app/api/llm/[promptId]/route.ts: verifies the Firebase ID token, then calls llm.call(). This is the only way clients reach an LLM.
 5. app/api/grade/route.ts: verifies the token and grades submitted answers server-side against questions (never send correctIndex to the client). Returns isCorrect, explanation and distractorTag per answer.
 6. Seeds (add them to seed.ts):
 - Quick assessment: 10 aptitude, 10 coding/code-reasoning, 10 communication (communication = spoken/typed prompts from commPrompts).
 - Mock interview: 10 timed technical/aptitude MCQs + 2 coding problems (statement, constraints, examples).
 - Double-check every answer key.
 7. The app layout with a sidebar matching our sketch: Personal Details, Quick Assessment, Skill Gap, Roadmap, Progress, Mock Interview, Daily Practice, Projects, Placement Readiness, Connect. Create empty pages for each route.
- Run typecheck, build and seed. Report the files you created and the repository function names, so the other tracks can use them.

Message: Read PLAN.md, README.md, lib/schemas, lib/data, lib/auth.tsx and lib/engine. Firebase is the backend; use only the lib/data functions and the /api/llm and /api/grade routes. Do not edit lib/schemas, lib/data, lib/llm/call.ts or other tracks' pages; if you need a change there, stop and tell me. Polished shadcn UI, with loading and error states on every LLM call. When done, report the files you changed and how to test.

Pasted: Track A

Build:

1. Login: Google Sign-In. New users go to onboarding; partially onboarded users resume at their saved onboardingStep; finished users go to /home.
2. Onboarding (PLAN.md Section 1, point 1): target role, up to 3 companies, placement date, hours per day.
3. /home: readiness headline per target company, a "Today" card (the next roadmap task, or "Take Quick Assessment"), and a module grid.
4. /personal-details: view and edit the profile.
5. /connect: GitHub, LinkedIn and LeetCode URLs plus resume upload to Firebase Storage (store only the URLs/paths).
6. /readiness: per company, component bars against the company bars, the weights table, the arithmetic shown, the biggest lever and the confidence label, using lib/engine/readiness.ts.
7. /progress: a Recharts line chart of readinessSnapshots over time per company, plus category score history.

Pasted: Track B

Build:

1. /assessment, matching our sketch: topic tabs (Aptitude x/10, Coding x/10, Communication x/10) with a progress bar, the question on the left, a Q1...Q30 palette showing answered/unanswered on the right, and Submit.

- MCQs: require a confidence chip (Sure / Think so / Guessing) before an answer is accepted, and track time per question.

- Communication: a textarea plus an optional mic button using the browser Web Speech API (fall back to typing if unsupported). Score each answer with P8 (implement it per PLAN.md Section 6; compute WPM and fillers in code where timing exists).

- On submit: /api/grade → save attempts → classifierRules on the wrong answers (P4 LLM only for ambiguous cases) → save mistakes → mastery updates → categoryScores → readiness snapshot.

2. /skill-gap, matching our sketch: overall % ring plus Coding / Aptitude / Communication rings.

- "Where I stand": scores against the target companies' bars.

- "What I lag behind": weakest skills plus the mistake-type breakdown (concept / application / careless / overconfident), with an explanation for each.

- "Suggestions for your role and companies": a new prompt P-GAP returning { mustLearn: [{skill, why}], shouldHave: [{item, why}], companyExpectations: [{companyId, expects: string[], yourGap}] }. Save it to Firestore.

3. /daily-practice: a 10-question set picked from the user's weakest skills and open mistakes, reusing the assessment runner. On finish, update mastery, readiness and a snapshot, and show the before → after change.

Pasted: Track C

Build /mock-interview, matching our sketch:

1. Round 1, Technical & Aptitude: 10 MCQs, a visible 10-minute countdown that auto-submits, graded via /api/grade, then "Next Round →".

2. Round 2, Coding: the problem statement, constraints and examples; a code workspace (monospace editor plus a language select; no execution); a 20-minute timer. Submit calls a new prompt P-CODE: input problem + code + language → output { likelyCorrect, issues: [{line, problem}], timeComplexity, spaceComplexity, edgeCasesMissed: [], score: 0–10, hint }. Label it clearly as an AI review, not an execution.

3. Round 3, Project Defense: pick a project from the user's Firestore projects (Person D builds those); if there are none, paste a description. P6 asks 5 questions in chat style, unsupported claims first; then P7 evaluates. Write defenseScore back to the project.

4. A final report with per-round scores, strengths and gaps. Save it as an interviewSession and update the readiness snapshot.

Pasted: Track D

Build:

1. /roadmap, matching our sketch: code computes the budget (days left until placementDate; minutes per category per day from hoursPerDay, weighted by company weight × gap). P5 sequences the tasks, mapping concept → learn, application → mixed drill, careless → timed drill, overconfident → calibration drill. UI: Today cards such as "Coding practice — 30 mins", each with a reason; a week view; tick tasks as done; a "Regenerate plan" button. Without an assessment, show a call to action to take it.
2. /projects: add a project from a GitHub URL (public repos; fetch the README, file tree and up to 8 key files through a server route with Octokit and a GITHUB_TOKEN) and/or from the resume (P1 on the uploaded PDF). Run P3 → a claims table (claim → verdict → evidence file refs) and interview hooks. Save to Firestore. If the GitHub fetch fails, fall back to resume-only with claims marked "unverified".

Message: All tracks are merged. Fix typecheck and build errors; make every sidebar link work; make sure nothing crashes for a brand-new user with no data. Check that the loop works end to end: assessment → skill gap → roadmap → daily practice → readiness/progress update. Then create a seeded demo user (scripts/seed-demo.ts) with an assessment, mistakes, a roadmap, one analyzed project, one interview session and 5 readiness snapshots, so every page shows data for judges. Update README.md: problem, features, differentiators, the readiness formula and the list of prompts. Report what you fixed.

Your choice: Build it all here now · Keep server-side writes.

14–16. Auth, Gemini and daily practice

14. Google sign-in is not enabled for this Firebase project yet. lets do this
15. in daily practice , i need some coding level question based on what doamin the user has chosen ,for each domain ,the approach on diiferent algorithms and patterns should be there .AI is not configured on this server (ANTHROPIC_API_KEY missing). use gemini key : [REDACTED]
16. daily practice , in the sense daily coding only from leetcode , codefroce , hackerramk and many ohter things , the code question should be taken and given the user shold type the code and submit

Your choice: Problems: curated classics + links · Judging: run tests in browser + AI review.

17–20. Deployment

17. project in Cloud run , if you cant deploy on it , use vercel , are we runing it in firebase ?so no need vercel right ?

Your choice: Vercel now (free, no card).

18. can we do it in github ?
19. ok vercel acc created
20. from here itself do it in vercel

21–23. Documentation

21. ok , now explain me the project implementation plan , and the import parts in the project .What each code file does .
22. change the plan , README to pdf and give
23. put all the prompts i gave u inn a pdf and give me